Chapitre 4 — Fonctions

> **Partie I — Fondations du C++ moderne**
> Norme de référence : C++20 / C++23 — Compilateurs : GCC 14+, Clang 18+, MSVC 2022 (17.10+)

4.1 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de :

1. Déclarer, définir et appeler une fonction, et distinguer déclaration et définition.
2. Expliquer ce qui se passe en mémoire lors d'un appel (pile d'appels, cadre d'appel).
3. Utiliser les références et choisir le bon mode de passage des paramètres (valeur, référence, référence constante).
4. Renvoyer un ou plusieurs résultats proprement (structure, `std::pair`, `std::tuple`, liaisons structurées).
5. Utiliser les arguments par défaut et la surcharge, et prévoir quelle surcharge sera appelée.
6. Écrire des fonctions `inline`, `constexpr` et `consteval`, et utiliser les attributs `[[nodiscard]]`, `[[maybe_unused]]`, `[[deprecated]]`.
7. Écrire des fonctions récursives et en connaître les limites.
8. Écrire et utiliser des lambdas, avec les différents modes de capture.
9. Organiser le code en espaces de noms.
10. Documenter et vérifier les préconditions d'une fonction.
11. Concevoir des fonctions claires, courtes et faciles à tester.

4.2 Pourquoi des fonctions ?

Une **fonction** est un bloc de code nommé, qui réalise une tâche précise, et que l'on peut appeler autant de fois que nécessaire. Les fonctions permettent :

Avantage	Explication
Décomposer	Un problème complexe est découpé en sous-problèmes simples, chacun traité par une fonction.
Réutiliser	Un calcul écrit une fois (la moyenne) sert partout, sans copier-coller.
Abstraire	L'appelant sait **ce que** fait la fonction, pas **comment** : `calculerMoyenne(notes)` se lit comme une phrase.
Maintenir	Une correction dans la fonction profite à tous les appels.
Tester	Une fonction isolée se teste facilement (chapitre 23).

> 💡 **Signe qu'il faut créer une fonction** : un bloc de code copié-collé, un commentaire qui annonce « ici on calcule… », ou une fonction qui dépasse une quarantaine de lignes.

4.3 Anatomie d'une fonction

4.3.1 Syntaxe

```
double moyenneDeDeux(double a, double b)     // en-tête
{                                            // corps
    return (a + b) / 2.0;
}
```

Élément	Dans l'exemple	Rôle
Type de retour	`double`	Type de la valeur renvoyée
Nom	`moyenneDeDeux`	Identifiant de la fonction
Paramètres	`double a, double b`	Variables locales qui reçoivent les valeurs transmises
Corps	`{ ... }`	Instructions exécutées
`return`	`return (a + b) / 2.0;`	Termine la fonction et renvoie une valeur

Appel :

```
double m = moyenneDeDeux(12.0, 15.0);   // 12.0 et 15.0 sont les ARGUMENTS
```

On distingue les **paramètres** (dans la définition : `a`, `b`) et les **arguments** (valeurs fournies à l'appel : `12.0`, `15.0`).

4.3.2 La signature

La **signature** d'une fonction est constituée de son **nom** et de la **liste des types de ses paramètres**. Le type de retour n'en fait pas partie. C'est la signature qui permet au compilateur de distinguer des fonctions de même nom (surcharge, section 4.10).

4.3.3 Fonctions sans résultat : `void`

```cpp
void afficherSeparateur(int longueur)
{
    std::println("{}", std::string(static_cast<std::size_t>(longueur), '-'));
}
```

Une fonction `void` peut contenir `return;` (sans valeur) pour se terminer plus tôt.

4.3.4 Plusieurs `return`

Une fonction peut contenir plusieurs `return`. Le style dit **« de sortie anticipée »** (*early return*) traite d'abord les cas particuliers, ce qui évite les imbrications profondes :

```cpp
std::string_view decision(double moyenne)
{
    if (moyenne < 0.0 || moyenne > 20.0) {
        return "Moyenne invalide";
    }
    if (moyenne < 10.0) {
        return "Ajourné";
    }
    return "Admis";
}
```

> Une fonction dont le type de retour n'est pas `void` **doit** renvoyer une valeur sur **tous** les chemins. Atteindre la fin sans `return` est un comportement indéfini (sauf pour `main`). Le compilateur le signale avec `-Wall` (`-Wreturn-type`).

4.3.5 La fonction `main`

`main` est une fonction particulière : point d'entrée du programme, elle ne peut être ni appelée par le programme, ni surchargée. Elle existe sous deux formes :

```
int main() { /* ... */ }

int main(int argc, char* argv[]) { /* ... */ }
```

La seconde forme reçoit les **arguments de la ligne de commande** : `argc` en donne le nombre (le nom du programme compris), `argv` les contient sous forme de chaînes C.

```
#include <cstdlib>
#include <print>
#include <string_view>

int main(int argc, char* argv[])
{
    if (argc > 1 && std::string_view{argv[1]} == "--aide") {
        std::println("Usage : bulletin [--aide]");
        return EXIT_SUCCESS;
    }
    // ...
    return EXIT_SUCCESS;
}
```

```
./bulletin --aide
```

La valeur renvoyée par `main` est transmise au système : `0` ou `EXIT_SUCCESS` signalent un succès, une autre valeur (souvent `EXIT_FAILURE`) un échec. Les scripts et les outils d'intégration continue s'en servent. Les pointeurs (`char*`) seront étudiés au chapitre 5.

4.4 Déclaration et définition

4.4.1 Deux notions différentes

- La **déclaration** (ou **prototype**) annonce la fonction : son type de retour, son nom, ses paramètres. Elle se termine par un point-virgule.
- La **définition** fournit en plus le **corps**.

```
double moyenneDeDeux(double a, double b);          // déclaration

int main()
{
    double m = moyenneDeDeux(12.0, 15.0);          // OK : la fonction est déclarée
}

double moyenneDeDeux(double a, double b)           // définition
{
    return (a + b) / 2.0;
}
```

Le compilateur lit le fichier **de haut en bas** : une fonction doit être **déclarée avant d'être appelée**. Sa définition peut venir plus loin, ou dans un autre fichier.

4.4.2 Une définition, plusieurs déclarations

Une fonction peut être déclarée plusieurs fois, mais **définie une seule fois** dans tout le programme : c'est la **règle de la définition unique** (*One Definition Rule*, ODR), détaillée au chapitre 6. En pratique :

- les **déclarations** vont dans un fichier d'en-tête (`.hpp`), inclus partout où la fonction est utilisée ;
- la **définition** va dans un fichier source (`.cpp`), compilé une seule fois.

C'est exactement l'organisation adoptée pour UIN-Campus depuis le chapitre 2.

> Dans une déclaration, les noms des paramètres sont facultatifs (`double moyenneDeDeux(double, double);`), mais il est recommandé de les écrire : ils documentent la fonction.

4.5 Ce qui se passe lors d'un appel

4.5.1 La pile d'appels

À chaque appel de fonction, le programme réserve sur la **pile** (*stack*) une zone appelée **cadre d'appel** (*stack frame*). Elle contient les paramètres, les variables locales et l'adresse où revenir après l'appel. À la fin de la fonction, le cadre est **libéré** et toutes ses variables locales sont détruites.

```
double carre(double x)
{
    double r = x * x;
    return r;
}

double sommeDesCarres(double a, double b)
{
    return carre(a) + carre(b);
}

int main()
{
    double s = sommeDesCarres(3.0, 4.0);
}
```

État de la pile pendant le premier appel de `carre` :

```
        ┌──────────────────────────────┐
sommet  │ carre                        │  x = 3.0, r = 9.0
        ├──────────────────────────────┤
        │ sommeDesCarres               │  a = 3.0, b = 4.0
        ├──────────────────────────────┤
base    │ main                         │  s = (pas encore calculé)
        └──────────────────────────────┘
```

C'est exactement cette pile que la commande `backtrace` du débogueur affiche (chapitre 2).

4.5.2 Conséquences importantes

- Les **paramètres sont des variables locales** de la fonction, initialisées avec les arguments.
- Les variables locales **n'existent plus** après le retour de la fonction : il ne faut jamais renvoyer une référence ou un pointeur vers elles (section 4.8.2).
- La pile a une **taille limitée** (souvent 1 à 8 Mo) : une récursion trop profonde la fait déborder (section 4.16).

4.6 Les références

Avant d'étudier le passage des paramètres, il faut connaître les **références**, l'une des différences majeures entre le C et le C++.

4.6.1 Définition

Une **référence** est un **autre nom** (un alias) pour une variable existante.

```
int note = 12;
int& ref = note;     // ref est un autre nom pour note

ref = 15;            // modifie note
std::println("{}", note);   // 15
```

`ref` n'est pas une copie : c'est **la même variable** sous un autre nom.

4.6.2 Règles

- Une référence **doit être initialisée** à sa déclaration : `int& r;` ne compile pas.
- Une référence **ne peut pas être redirigée** : une fois liée à `note`, elle le reste. `ref = autre;` copie la valeur de `autre` dans `note`, il ne change pas la cible de `ref`.

- Il n'existe pas de « référence nulle ».

4.6.3 Référence constante

Une **référence constante** (`const T&`) permet de lire la variable **sans pouvoir la modifier** :

```
std::string nom{"Aïssata Moussa"};
const std::string& lecture = nom;
std::println("{}", lecture.size());   // OK : lecture
lecture += "!";                       // ERREUR : lecture seule
```

Propriété importante : une référence constante peut se lier à une **valeur temporaire**, alors qu'une référence non constante ne le peut pas.

```
const std::string& a = "texte";   // OK : un std::string temporaire est créé et sa durée de vie prolongée
std::string& b = "texte";         // ERREUR
const double& c = 3.14;           // OK
```

C'est cette propriété qui permet d'appeler une fonction prenant un `const std::string&` avec un littéral.

> Les références `&` étudiées ici sont des **références lvalue**. Le C++11 a aussi introduit les **références rvalue** `&&` , base de la sémantique de déplacement (chapitre 9).

4.7 Le passage des paramètres

4.7.1 Passage par valeur

Le paramètre reçoit une **copie** de l'argument. Modifier le paramètre ne change pas l'argument.

```
void bonifier(double note)
{
    note += 1.0;            // modifie la copie locale seulement
}

double n = 12.0;
bonifier(n);
std::println("{}", n);      // 12 : n n'a pas changé
```

4.7.2 Passage par référence

Le paramètre est un **alias** de l'argument : la fonction peut le **modifier**.

```
void bonifier(double& note)
{
    note = std::min(note + 1.0, 20.0);
}

double n = 12.0;
bonifier(n);
std::println("{}", n);      // 13
bonifier(12.0);             // ERREUR : un littéral ne peut pas être modifié
```

4.7.3 Passage par référence constante

Le paramètre est un alias **en lecture seule** : pas de copie, et aucune modification possible.

```cpp
double moyenne(const std::vector<double>& notes)   // aucune copie du vecteur
{
    double somme{0.0};
    for (double n : notes) {
        somme += n;
    }
    return somme / static_cast<double>(notes.size());
}
```

Si ce paramètre était passé **par valeur** (`std::vector<double> notes`), chaque appel **copierait tout le vecteur** : pour 10 000 notes, 10 000 copies de `double` et une allocation mémoire, à chaque appel, pour rien.

4.7.4 Quel mode choisir ?

Les **C++ Core Guidelines** (règles F.15 à F.18) donnent une méthode simple, fondée sur le **rôle** du paramètre :

Rôle du paramètre	**Type « petit » et bon marché à copier (`int` , `double` , `bool` , `char` , `std::string_view` , énumérations)**	**Type coûteux à copier (`std::string` , `std::vector` , objets)**
Entrée (lue seulement)	Par valeur : `f(double x)`	Par référence constante : `f(const std::string& s)`
Entrée-sortie (lue et modifiée)	Par référence : `f(int& x)`	Par référence : `f(std::vector<double>& v)`
Sortie (seulement produite)	**Renvoyer la valeur** plutôt qu'un paramètre	**Renvoyer la valeur**
Puits (la fonction garde	Par valeur	Par valeur, puis `std::move` (chapitre 9)

Rôle du paramètre	Type « petit » et bon marché à copier (`int`, `double`, `bool`, `char`, `std::string_view`, énumérations)	Type coûteux à copier (`std::string`, `std::vector`, objets)
une copie)		

Exemples :

```
bool estNoteValide(double note);                          // entrée, petit type : valeur
void afficherBulletin(const std::string& nom,
                      const std::vector<double>& notes);  // entrées coûteuses : const&
void appliquerBonus(std::vector<double>& notes, double bonus);   // entrée-sortie : &
std::vector<double> lireNotes(int nombre);                // sortie : valeur de retour
```

> 💡 **Règle simple à retenir** :
>
> - petit type → **par valeur** ;
> - gros type en lecture → **`const&`** ;
> - besoin de modifier l'argument → **`&`** ;
> - résultat → **`return`** .

> **Passer un `int` ou un `double` par `const&`** n'apporte rien : une référence coûte autant qu'une copie de ces petits types, et ajoute une indirection. Réservez `const&` aux types coûteux.

4.7.5 `const` sur un paramètre par valeur

```
double arrondir(const double valeur, const int decimales);
```

Ce `const` ne concerne que la **copie locale** : il n'a aucun effet pour l'appelant. On peut l'ajouter dans la **définition** pour s'interdire de modifier le

paramètre par erreur, mais il est inutile dans la **déclaration**.

4.8 Les valeurs de retour

4.8.1 Retour par valeur

Renvoyer un objet par valeur, même volumineux, est **efficace** en C++ moderne. Le compilateur construit directement le résultat à sa destination finale, sans copie : c'est l'**élision de copie** (*RVO, Return Value Optimization*), garantie dans de nombreux cas depuis C++17. Quand elle ne s'applique pas, l'objet est **déplacé** plutôt que copié (chapitre 9).

```
std::vector<double> genererNotes(int n)
{
    std::vector<double> notes;
    for (int i{0}; i < n; ++i) {
        notes.push_back(10.0);
    }
    return notes;          // pas de copie du vecteur
}
```

> 💡 **Préférez renvoyer un résultat** plutôt que de le produire dans un paramètre de sortie (`void genererNotes(int n, std::vector<double>& resultat)`). Le code est plus lisible et le résultat peut être `const` .

4.8.2 Le piège de la référence pendante

```
const std::string& titre()
{
    std::string t{"Bulletin"};
    return t;              //  t est détruite à la sortie : référence pendante
}
```

Renvoyer une **référence** (ou un pointeur) vers une **variable locale** est un comportement indéfini : la variable n'existe plus quand l'appelant l'utilise. Le compilateur le signale (`-Wreturn-local-addr`), et AddressSanitizer le détecte à l'exécution.

Renvoyer une référence est légitime seulement quand l'objet référencé **survit** à la fonction : un paramètre reçu par référence, ou un attribut d'objet (chapitre 7).

```
const std::string& plusLong(const std::string& a, const std::string& b)
{
    return (a.size() >= b.size()) ? a : b;    // OK : a et b appartiennent à l'appelant
}
```

4.8.3 Renvoyer plusieurs valeurs

Une fonction ne renvoie qu'**une** valeur, mais cette valeur peut en regrouper plusieurs.

Solution 1 — une structure (recommandée)

```cpp
struct Extremes {
    double minimum{};
    double maximum{};
};

Extremes calculerExtremes(const std::vector<double>& notes)
{
    Extremes e{notes.front(), notes.front()};
    for (double n : notes) {
        e.minimum = std::min(e.minimum, n);
        e.maximum = std::max(e.maximum, n);
    }
    return e;
}
```

Une `struct` regroupe des données nommées ; nous l'utilisons ici comme simple « paquet » de valeurs, les structures et classes seront étudiées en détail au chapitre 7. Les champs ont des **noms explicites** : c'est ce qui la rend préférable aux solutions suivantes (Core Guidelines F.21).

Solution 2 — `std::pair` (`<utility>`) pour deux valeurs :

```cpp
std::pair<double, double> extremes(const std::vector<double>& notes);

auto p = extremes(notes);
std::println("{} {}", p.first, p.second);     // first ? second ? peu parlant
```

Solution 3 — `std::tuple` (`<tuple>`) pour plusieurs valeurs :

```cpp
std::tuple<double, double, double> statistiques(const std::vector<double>& notes);

auto t = statistiques(notes);
double moy = std::get<2>(t);                // encore moins parlant
```

4.8.4 Les liaisons structurées (C++17)

Les **liaisons structurées** (*structured bindings*) décomposent directement une structure, une paire, un tuple ou un tableau en variables nommées :

```cpp
const auto [mini, maxi] = calculerExtremes(notes);
std::println("Min : {}, Max : {}", mini, maxi);

auto [premier, second] = std::pair{1, 2};

const auto [a, b, c] = statistiques(notes);
```

Les noms sont choisis librement à l'appel, dans l'**ordre** des membres. On peut aussi écrire `auto&` ou `const auto&` pour obtenir des références plutôt que des copies :

```cpp
Extremes e = calculerExtremes(notes);
auto& [mn, mx] = e;       // mn et mx sont des alias de e.minimum et e.maximum
mx = 20.0;                // modifie e.maximum
```

Les liaisons structurées sont très utilisées pour parcourir les dictionnaires (chapitre 18) :

```cpp
for (const auto& [matricule, moyenne] : moyennesParMatricule) { /* ... */ }
```

> Style ancien équivalent, à reconnaître : `std::tie(mini, maxi) = extremes(notes);` (variables déclarées au préalable).

4.9 Les arguments par défaut

Un paramètre peut recevoir une **valeur par défaut**, utilisée quand l'argument est omis :

```
double arrondir(double valeur, int decimales = 2);

arrondir(13.456);       // 13.46 : decimales vaut 2
arrondir(13.456, 1);    // 13.5
```

Règles :

- les paramètres par défaut sont **en fin de liste** : `void f(int a = 1, int b);` est interdit ;
- on ne peut omettre un argument qu'en omettant **tous ceux qui suivent** : avec `f(int a, int b = 2, int c = 3)`, on ne peut pas fournir `c` sans fournir `b` ;
- la valeur par défaut s'écrit **une seule fois**, dans la **déclaration** (donc dans l'en-tête), pas dans la définition.

```cpp
// arrondi.hpp
double arrondir(double valeur, int decimales = 2);

// arrondi.cpp
double arrondir(double valeur, int decimales)    // pas de répétition de = 2
{
    const double facteur = std::pow(10.0, decimales);
    return std::round(valeur * facteur) / facteur;
}
```

> Les arguments par défaut peuvent créer des **ambiguïtés** avec la surcharge : si `f(int)` et `f(int, int = 0)` coexistent, l'appel `f(5)` est ambigu et ne compile pas.

4.10 La surcharge de fonctions

4.10.1 Principe

On peut définir **plusieurs fonctions de même nom**, à condition que leurs **listes de paramètres** diffèrent (en nombre ou en types). C'est la **surcharge** (*overloading*).

```
double moyenne(const std::vector<double>& notes);                        // moyenne simple
double moyenne(const std::vector<double>& notes,
               const std::vector<double>& coefficients);                 // moyenne pondérée
void afficher(int valeur);
void afficher(double valeur);
void afficher(std::string_view texte);
```

À l'appel, le compilateur choisit la bonne version d'après les arguments. Le **type de retour seul ne suffit pas** à distinguer deux surcharges :

```
int    lire();
double lire();          // ERREUR : ne diffère que par le type de retour
```

4.10.2 La résolution de surcharge

Pour choisir, le compilateur classe les conversions nécessaires pour chaque candidate, de la meilleure à la moins bonne :

Rang	Conversion	Exemples
1	**Correspondance exacte**	`int` → `int`, tableau → pointeur, ajout de `const`
2	**Promotion**	`char`, `short`, `bool` → `int` ; `float` → `double`
3	**Conversion standard**	`int` → `double`, `double` → `int`, `long` → `int`
4	**Conversion définie par l'utilisateur**	`const char*` → `std::string` (constructeur, chapitre 8)

La candidate qui demande les **meilleures conversions** l'emporte. Si deux candidates sont aussi bonnes l'une que l'autre, l'appel est **ambigu** : erreur de compilation.

Avec `void f(int);` et `void f(double);` :

Appel	Fonction choisie	Raison
`f(5)`	`f(int)`	Correspondance exacte
`f(5.0)`	`f(double)`	Correspondance exacte
`f(5.0f)`	`f(double)`	Promotion `float` → `double`
`f('a')`	`f(int)`	Promotion `char` → `int`
`f(true)`	`f(int)`	Promotion `bool` → `int`
`f(5L)`	**Ambigu**	`long` → `int` et `long` → `double` sont deux conversions standard

4.10.3 Surcharge et références

On peut surcharger selon qu'un paramètre est une référence constante ou non :

```
void traiter(const std::string& s);   // appelée pour les constantes et les temporaires
void traiter(std::string& s);         // appelée pour les variables modifiables
```

Cette technique prendra tout son sens avec les références rvalue (chapitre 9).

4.10.4 Interdire une surcharge avec `= delete`

Les conversions implicites peuvent laisser passer des erreurs. On peut **interdire** explicitement une version :

```
void attribuerCredits(int credits);
void attribuerCredits(double) = delete;   // interdit de passer un réel

attribuerCredits(6);      // OK
attribuerCredits(4.5);    // ERREUR : fonction supprimée (sans cela, 4.5 serait tronqué en 4)
```

> 💡 **Bonne pratique** : une surcharge doit réaliser **la même opération** sur des types différents. Si deux fonctions font des choses différentes, donnez-leur des noms différents.

4.11 Les fonctions `inline`

Historiquement, `inline` était une **suggestion** d'optimisation : remplacer l'appel par le corps de la fonction pour éviter le coût de l'appel. Aujourd'hui, les compilateurs décident seuls de ce remplacement, bien mieux que le programmeur, et ignorent cette suggestion.

Le sens actuel de `inline` concerne la **règle de la définition unique** : une fonction `inline` peut être **définie dans un en-tête** inclus dans plusieurs fichiers `.cpp`, sans provoquer d'erreur « définition multiple » à l'édition des liens.

```
// outils.hpp
inline double pourcentage(double partie, double total)
{
    return 100.0 * partie / total;
}
```

Sont implicitement `inline` : les fonctions `constexpr` et `consteval`, et les méthodes définies à l'intérieur d'une classe (chapitre 7).

4.12 Fonctions `constexpr` et `consteval`

4.12.1 Fonctions `constexpr`

Une fonction `constexpr` **peut** être exécutée par le compilateur, si ses arguments sont connus à la compilation. Sinon, elle s'exécute normalement.

```
constexpr long long factorielle(int n)
{
    long long resultat{1};
    for (int i{2}; i <= n; ++i) {
        resultat *= i;
    }
    return resultat;
}

constexpr long long f10 = factorielle(10);       // calculé à la compilation
static_assert(factorielle(5) == 120);            // vérifié à la compilation

int n{};
std::cin >> n;
long long fn = factorielle(n);                   // calculé à l'exécution
```

`static_assert(condition)` vérifie une condition **à la compilation** : si elle est fausse, le programme ne compile pas. C'est une forme de test gratuite.

Depuis C++20, presque tout le langage est permis dans une fonction `constexpr` : boucles, variables locales, `if`, `switch`, et même `std::vector` et `std::string` (à condition qu'ils soient détruits avant la fin de l'évaluation).

Intérêts :

- **performance** : le calcul est fait une seule fois, par le compilateur ;
- **sûreté** : un comportement indéfini rencontré pendant une évaluation à la compilation provoque une **erreur de compilation** ;
- **tests à la compilation** avec `static_assert`.

4.12.2 Fonctions `consteval` et `if consteval`

Rappel du chapitre 3 : une fonction `consteval` **doit** être évaluée à la compilation. Le C++23 ajoute `if consteval`, qui permet à une fonction `constexpr` de choisir un code différent selon qu'elle est évaluée à la compilation ou à l'exécution. Ces outils relèvent de l'usage avancé.

4.13 Les attributs

Les **attributs** (`[[...]]`) donnent des informations supplémentaires au compilateur.

4.13.1 `[[nodiscard]]`

Signale qu'**ignorer le résultat** de la fonction est probablement une erreur :

```
[[nodiscard]] double moyenne(const std::vector<double>& notes);

moyenne(notes);    // AVERTISSEMENT : résultat ignoré
```

Depuis C++20, on peut en préciser la raison :

```cpp
[[nodiscard("la moyenne calculée doit être utilisée")]]
double moyenne(const std::vector<double>& notes);
```

> 💡 **Bonne pratique** : marquez `[[nodiscard]]` toute fonction dont l'**unique intérêt** est sa valeur de retour (calculs, conversions, tests `estValide` …).

4.13.2 Autres attributs utiles

Attribut	Rôle	Exemple
`[[maybe_unused]]`	Supprime l'avertissement « variable ou paramètre inutilisé »	`void f([[maybe_unused]] int debug);`
`[[deprecated("raison")]]`	Signale qu'une fonction est obsolète	`[[deprecated("utiliser moyenne()")]] double calcMoy(...);`
`[[noreturn]]`	La fonction ne revient jamais (elle termine le programme ou lance une exception)	`[[noreturn]] void erreurFatale(std::string_view msg);`
`[[fallthrough]]`	Passage volontaire au `case` suivant (chapitre 3)	
`[[likely]]`, `[[unlikely]]`	Branche probable ou improbable (chapitre 3)	

Le mot-clé `noexcept`, qui indique qu'une fonction ne lance pas d'exception, sera étudié aux chapitres 9 et 15.

4.14 Déduction du type de retour

4.14.1 `auto` comme type de retour (C++14)

Le compilateur peut déduire le type de retour à partir des instructions `return` :

```
auto pourcentage(double partie, double total)     // type déduit : double
{
    return 100.0 * partie / total;
}
```

Tous les `return` doivent produire **le même type**. Cette forme est pratique pour les fonctions courtes et en programmation générique, mais un type explicite documente mieux l'interface d'une fonction publique.

4.14.2 Type de retour placé à la fin

Une autre syntaxe place le type de retour **après** les paramètres :

```
auto pourcentage(double partie, double total) -> double;
```

Elle est nécessaire dans certains cas génériques, quand le type de retour dépend des paramètres (chapitre 16), et on la rencontre dans certaines bases de code par souci d'uniformité.

4.15 Variables locales statiques

Une variable locale déclarée `static` est initialisée **une seule fois**, au premier passage, et **conserve sa valeur** entre les appels :

```cpp
int prochainNumero()
{
    static int compteur{0};    // initialisé une seule fois
    return ++compteur;
}

prochainNumero();   // 1
prochainNumero();   // 2
prochainNumero();   // 3
```

Depuis C++11, cette initialisation est garantie correcte même avec plusieurs threads.

> **Attention** : une variable statique est un **état caché**. La fonction ne renvoie plus toujours le même résultat pour les mêmes arguments, ce qui la rend plus difficile à comprendre et à tester. Nous verrons au chapitre 10 de meilleures façons de partager un état (membres statiques de classe).

4.16 La récursivité

4.16.1 Principe

Une fonction **récursive** s'appelle elle-même. Elle comporte toujours :

- un **cas de base**, qui s'arrête sans appel récursif ;
- un **cas récursif**, qui se ramène à un problème **plus petit**.

```cpp
constexpr long long factorielle(int n)
{
    if (n <= 1) {                       // cas de base
        return 1;
    }
    return n * factorielle(n - 1);      // cas récursif
}
```

Déroulement de `factorielle(4)` :

```
factorielle(4) = 4 * factorielle(3)
                     = 3 * factorielle(2)
                           = 2 * factorielle(1)
                                 = 1              ← cas de base
                           = 2 * 1 = 2
                     = 3 * 2 = 6
               = 4 * 6 = 24
```

4.16.2 Exemple : l'algorithme d'Euclide

```cpp
constexpr int pgcd(int a, int b)
{
    return (b == 0) ? a : pgcd(b, a % b);
}

static_assert(pgcd(48, 18) == 6);
```

4.16.3 Limites de la récursivité

- **Débordement de pile** : chaque appel ajoute un cadre sur la pile. Une récursion trop profonde (par exemple sur un million d'éléments) provoque un *stack overflow* et le plantage du programme.
- **Explosion du nombre d'appels** : la version naïve de Fibonacci recalcule sans cesse les mêmes valeurs.

```
long long fibonacci(int n)      //  complexité exponentielle
{
    if (n < 2) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}
// fibonacci(45) demande plus d'un milliard d'appels
```

Version itérative, en temps linéaire :

```
long long fibonacci(int n)
{
    long long precedent{0};
    long long courant{1};
    for (int i{0}; i < n; ++i) {
        const long long suivant = precedent + courant;
        precedent = courant;
        courant = suivant;
    }
    return precedent;
}
```

> 💡 **Quand utiliser la récursivité ?** Quand la structure du problème est elle-même récursive : parcours d'arbres, algorithmes « diviser pour régner » (tri rapide, recherche dichotomique), analyse d'expressions. Pour une simple répétition, préférez une boucle.

4.17 Les lambdas

4.17.1 Motivation

De nombreux algorithmes de la bibliothèque standard attendent **une fonction en paramètre** : un critère de tri, une condition de recherche, une transformation. Écrire à chaque fois une fonction séparée serait lourd. Les **lambdas** (C++11) permettent d'écrire une fonction **anonyme, directement à l'endroit où on en a besoin**.

```
#include <algorithm>
#include <vector>

std::vector<double> notes{12.0, 8.5, 17.0, 14.25, 6.0};

// Trier par ordre décroissant
std::ranges::sort(notes, [](double a, double b) { return a > b; });

// Compter les notes supérieures ou égales à 10
auto nbAdmis = std::ranges::count_if(notes, [](double n) { return n >= 10.0; });
```

4.17.2 Syntaxe

```
[ captures ] ( paramètres ) -> type_de_retour { corps }
```

Partie	Rôle	Obligatoire ?
`[ ]`	**Captures** : variables extérieures utilisables dans la lambda	Oui (même vide)
`( )`	Paramètres, comme une fonction	Non si aucun paramètre
`-> type`	Type de retour	Non : il est généralement déduit
`{ }`	Corps	Oui

Une lambda est une **valeur** : on peut la stocker dans une variable (de type `auto`, car son type exact n'a pas de nom) et l'appeler comme une fonction.

```
auto carre = [](double x) { return x * x; };
std::println("{}", carre(4.0));    // 16
```

4.17.3 Les captures

Une lambda n'a **pas accès** aux variables locales qui l'entourent, sauf si elle les **capture** :

```
double seuil = 10.0;
auto estAdmis = [seuil](double n) { return n >= seuil; };   // seuil capturé par valeur
```

Capture	Signification
`[]`	Aucune capture
`[x]`	`x` capturée **par valeur** (copie, figée au moment de la création de la lambda)

Capture	Signification
`[&x]`	`x` capturée **par référence** (la lambda voit et peut modifier l'original)
`[x, &y]`	Mélange
`[=]`	Toutes les variables utilisées, par valeur
`[&]`	Toutes les variables utilisées, par référence
`[total = 0]`	**Capture avec initialisation** (C++14) : crée une variable propre à la lambda
`[this]`	L'objet courant (chapitre 7)

Capture par valeur ou par référence :

```
double bonus = 1.0;
auto parValeur    = [bonus](double n) { return n + bonus; };
auto parReference = [&bonus](double n) { return n + bonus; };

bonus = 2.0;
parValeur(10.0);      // 11 : copie faite à la création
parReference(10.0);   // 12 : voit la valeur actuelle
```

Modifier une variable extérieure par référence :

```cpp
int nbInvalides{0};
std::ranges::for_each(notes, [&nbInvalides](double n) {
    if (n < 0.0 || n > 20.0) {
        ++nbInvalides;
    }
});
```

Par défaut, une variable capturée par valeur est **en lecture seule** dans la lambda. Le mot-clé `mutable` autorise à modifier **la copie** :

```cpp
auto compteur = [n = 0]() mutable { return ++n; };
compteur();   // 1
compteur();   // 2
```

> 💡 **Bonne pratique** : capturez **explicitement** les variables nécessaires (`[seuil]` , `[&total]`) plutôt que `[=]` ou `[&]` . Le lecteur voit immédiatement de quoi dépend la lambda.

> **Piège : la capture par référence pendante.** Une lambda qui capture une variable locale par référence ne doit pas survivre à cette variable.

```cpp
auto creerFiltre()
{
    double seuil = 10.0;
    return [&seuil](double n) { return n >= seuil; };   //  seuil sera détruit
}
// Correction : capturer par valeur, [seuil]
```

4.17.4 Lambdas génériques (C++14)

Avec `auto` en paramètre, une lambda accepte des arguments de n'importe quel type :

```cpp
auto afficher = [](const auto& valeur) { std::println("{}", valeur); };
afficher(42);
afficher(3.14);
afficher("texte");
```

4.17.5 Initialiser une constante avec une lambda

Une lambda **appelée immédiatement** permet d'initialiser une variable `const` dont la valeur demande plusieurs instructions :

```cpp
const std::string mentionTexte = [moyenne] {
    if (moyenne >= 16.0) { return "Très bien"; }
    if (moyenne >= 14.0) { return "Bien"; }
    return "Autre";
}();                                  // les () finales appellent la lambda
```

4.17.6 Lambdas et algorithmes : aperçu

Algorithme (`<algorithm>`)	Rôle	Lambda attendue
`std::ranges::sort(v, comp)`	Trier	`(a, b) -> bool` : `a` doit-il précéder `b` ?
`std::ranges::count_if(v, pred)`	Compter	`(x) -> bool`
`std::ranges::find_if(v, pred)`	Chercher le premier élément vérifiant une condition	`(x) -> bool`
`std::ranges::any_of(v, pred)` , `all_of` ,	Tester	`(x) -> bool`

Algorithme (`<algorithm>`)	Rôle	Lambda attendue
`none_of`		
`std::ranges::for_each(v, f)`	Appliquer une action à chaque élément	`(x) -> void`
`std::ranges::transform(v, sortie, f)`	Transformer	`(x) -> y`

Ces algorithmes seront étudiés en profondeur aux chapitres 18 et 19. Retenez dès maintenant la démarche : **décrire ce que l'on veut** (« compter les notes ≥ 10 ») plutôt qu'écrire la boucle.

4.18 Pointeurs de fonction et `std::function` (aperçu)

Une fonction peut elle-même être stockée et transmise. Deux outils existent :

```cpp
#include <functional>

double addition(double a, double b) { return a + b; }
double maximum(double a, double b) { return std::max(a, b); }

// Pointeur de fonction (hérité du C)
using Operation = double (*)(double, double);
Operation op = &addition;
op(3.0, 4.0);                          // 7

// std::function : peut contenir une fonction, une lambda, un objet appelable
std::function<double(double, double)> f = maximum;
f = [](double a, double b) { return a * b; };
f(3.0, 4.0);                           // 12
```

`std::function` est plus souple, mais un peu plus coûteuse. Ces outils seront approfondis au chapitre 18 ; ils sont aussi à la base de patrons comme Strategy et Command (chapitre 22).

4.19 Les espaces de noms

4.19.1 Le problème

Dans un grand projet qui utilise plusieurs bibliothèques, deux fonctions peuvent porter le même nom : `calculer`, `afficher`, `Etudiant`… Les **espaces de noms** (*namespaces*) regroupent les noms pour éviter ces conflits.

```cpp
namespace uin {
    double moyenne(const std::vector<double>& notes);
}

namespace stats {
    double moyenne(const std::vector<double>& valeurs);
}

uin::moyenne(notes);      // pas de conflit
stats::moyenne(mesures);
```

L'opérateur `::` est l'**opérateur de résolution de portée**. `std` est l'espace de noms de la bibliothèque standard.

4.19.2 Définir un espace de noms

Un espace de noms peut être **ouvert plusieurs fois**, dans plusieurs fichiers : chaque bloc ajoute des déclarations au même espace.

```cpp
// regles.hpp
namespace uin {
    enum class Mention { /* ... */ };
}

// calculs.hpp
namespace uin {
    double moyenne(const std::vector<double>& notes);
}
```

Les espaces **imbriqués** s'écrivent de façon compacte depuis C++17 :

```cpp
namespace uin::calcul {       // équivalent à namespace uin { namespace calcul { ... } }
    double moyenne(const std::vector<double>& notes);
}

uin::calcul::moyenne(notes);
```

4.19.3 L'espace de noms anonyme

Un espace de noms **sans nom** rend ses éléments **invisibles en dehors du fichier** où il se trouve. On parle de **liaison interne** (*internal linkage*).

```cpp
// saisie.cpp
namespace {
    bool estReelStrict(std::string texte, double& valeur)   // fonction interne au fichier
    { /* ... */ }
}
```

C'est la manière moderne d'écrire des fonctions utilitaires **privées** à un fichier source (elle remplace l'ancien usage de `static` pour les fonctions). Deux fichiers peuvent ainsi avoir chacun leur fonction `estReelStrict` sans conflit à l'édition des liens.

4.19.4 `using` : déclaration et directive

Forme	Exemple	Effet
Déclaration `using`	`using std::string;`	Importe **un seul nom**
Directive `using`	`using namespace std;`	Importe **tous les noms** de l'espace

Forme	Exemple	Effet
Alias d'espace de noms	`namespace fs = std::filesystem;`	Nom court pour un espace long

Règles d'usage :

- **jamais** de `using namespace` dans un fichier d'en-tête : il s'imposerait à tous les fichiers qui l'incluent ;
- dans un fichier `.cpp`, limitez-les à une **portée locale** (dans une fonction) ;
- les alias (`namespace fs = std::filesystem;`) sont sans danger et très pratiques ;
- les directives ciblées comme `using namespace std::string_literals;` sont acceptables car elles n'importent que les suffixes de littéraux.

4.19.5 La recherche dépendante des arguments (ADL)

Pourquoi écrit-on `std::cout << x` sans écrire `std::operator<<(std::cout, x)` ? Grâce à la **recherche dépendante des arguments** (*Argument-Dependent Lookup*, ADL) : quand on appelle une fonction sans la qualifier, le compilateur la cherche aussi dans les espaces de noms des types de ses arguments. `std::cout` appartenant à `std`, l'opérateur `<<` est cherché dans `std`. Cette règle sera importante pour la surcharge des opérateurs (chapitre 11).

> Il est **interdit** d'ajouter ses propres déclarations dans l'espace de noms `std` (sauf cas très précis de spécialisation prévus par la norme).

4.20 Préconditions et postconditions

4.20.1 Le contrat d'une fonction

Une fonction bien conçue a un **contrat** :

- une **précondition** : ce qui doit être vrai **avant** l'appel (responsabilité de l'appelant) ;
- une **postcondition** : ce qui sera vrai **après** l'appel (responsabilité de la fonction).

Exemple : `moyenne(notes)` exige que `notes` ne soit pas vide (précondition) et garantit un résultat compris entre la plus petite et la plus grande note (postcondition).

4.20.2 Documenter et vérifier avec `assert`

```
#include <cassert>

/// Calcule la moyenne arithmétique des notes.
/// @pre notes n'est pas vide
[[nodiscard]] double moyenne(const std::vector<double>& notes)
{
    assert(!notes.empty() && "moyenne() : la liste de notes est vide");
    double somme{0.0};
    for (double n : notes) {
        somme += n;
    }
    return somme / static_cast<double>(notes.size());
}
```

`assert(condition)` arrête le programme avec un message indiquant le fichier et la ligne si la condition est fausse. L'astuce `&& "message"` ajoute une explication au diagnostic (une chaîne littérale est toujours « vraie »).

Points importants :

- `assert` est **désactivé** quand la macro `NDEBUG` est définie, c'est-à-dire dans les constructions `Release` (chapitre 2). Il sert à **détecter les erreurs de programmation** pendant le développement, pas à valider les saisies des utilisateurs ;

- pour les erreurs qui peuvent survenir en production (fichier absent, saisie invalide), on utilisera les exceptions ou `std::expected` (chapitre 15).

> Le C++26 introduit les **contrats** (`pre` , `post` , `contract_assert`), qui permettront d'écrire ces conditions directement dans la déclaration de la fonction.

4.21 Concevoir de bonnes fonctions

Principe	Explication
Une seule responsabilité	Une fonction fait **une** chose. `lireEtCalculerEtAfficher` doit devenir trois fonctions.
Un nom explicite	Un verbe pour une action (`calculerMoyenne` , `afficherBulletin`), une question pour un test (`estValide` , `aValide`).
Courte	Idéalement visible sans défilement. Au-delà d'une quarantaine de lignes, découper.
Peu de paramètres	Au-delà de 3 ou 4, regrouper les paramètres dans une structure.
Pas de paramètre booléen « drapeau »	`afficher(notes, true)` est illisible : que signifie `true` ? Préférer deux fonctions ou une énumération.
Préférer les fonctions pures	Une fonction **pure** ne dépend que de ses paramètres et ne modifie rien d'autre. Même entrée, même sortie : facile à comprendre et à tester.
Éviter les variables globales	Elles créent des dépendances cachées entre fonctions.
Séparer calcul et entrées-sorties	Une fonction qui calcule ne lit pas le clavier et n'affiche rien. On peut alors la tester et la réutiliser (interface graphique, fichier, réseau).

Exemple de paramètre drapeau et sa correction :

```
void afficherNotes(const std::vector<double>& notes, bool trier);   // afficherNotes(v, true) ?

enum class Ordre { Saisie, Croissant, Decroissant };
void afficherNotes(const std::vector<double>& notes, Ordre ordre);  // afficherNotes(v, Ordre::Croissant)
```

4.22 Erreurs fréquentes

Erreur	Conséquence	Correction
Appeler une fonction avant sa déclaration	Erreur de compilation	Déclarer avant (en-tête)
Oublier `return` dans une branche	Comportement indéfini	Activer `-Wall`, vérifier tous les chemins
Passer un vecteur ou une chaîne par valeur en lecture	Copies coûteuses à chaque appel	`const std::vector<T>&`
Passer un `int` par `const&`	Aucun gain, lisibilité réduite	Par valeur
Oublier le `&` pour modifier un argument	La modification est perdue	`T&`
Renvoyer une référence vers une variable locale	Référence pendante	Renvoyer par valeur
Répéter l'argument par défaut dans la définition	Erreur de compilation	Seulement dans la déclaration
Surcharges ambiguës	Erreur de compilation	Types plus distincts, ou `= delete`
Récursion sans cas de base	Débordement de pile	Vérifier la terminaison

Erreur	Conséquence	Correction
Lambda capturant une variable locale par référence et survivant à la fonction	Comportement indéfini	Capture par valeur
`using namespace` dans un en-tête	Conflits de noms partout	Qualifier avec `::`
Utiliser `assert` pour valider les saisies	Vérification absente en Release	Validation explicite, exceptions (chapitre 15)

4.23 Bonnes pratiques du chapitre

1. **Petits types par valeur, gros types par `const&`, modification par `&`, résultat par `return`.**
2. **Renvoyez des structures nommées** plutôt que des `std::pair` ou des paramètres de sortie, et récupérez-les avec les **liaisons structurées**.
3. **Marquez `[[nodiscard]]`** les fonctions dont le résultat ne doit pas être ignoré.
4. **Rendez `constexpr`** les fonctions de calcul qui peuvent l'être, et testez-les avec `static_assert`.
5. **Une fonction, une responsabilité** ; séparez calcul et entrées-sorties.
6. **Documentez les préconditions** et vérifiez-les avec `assert`.
7. **Capturez explicitement** dans les lambdas.
8. **Rangez votre code dans un espace de noms**, et les utilitaires internes dans un espace anonyme.
9. **Préférez les algorithmes avec lambdas** aux boucles écrites à la main quand ils expriment clairement l'intention.

4.24 Résumé du chapitre

- Une fonction se **déclare** (prototype) avant d'être appelée et se **définit** une seule fois.
- Chaque appel crée un **cadre** sur la **pile d'appels**, détruit au retour de la fonction.
- Une **référence** est un alias. Le **passage par référence constante** évite les copies en lecture ; le **passage par référence** permet de modifier l'argument.
- Le **retour par valeur** est efficace grâce à l'élision de copie. Pour plusieurs résultats, on renvoie une **structure** et on la décompose par **liaison structurée**.
- Les **arguments par défaut** se déclarent une fois, en fin de liste. La **surcharge** permet plusieurs fonctions de même nom, choisies par la **résolution de surcharge**.
- `inline` permet la définition dans un en-tête ; `constexpr` et `consteval` permettent le calcul à la compilation ; `[[nodiscard]]` protège les résultats importants.
- Une fonction **récursive** a un cas de base et un cas récursif ; attention à la profondeur de la pile.
- Les **lambdas** sont des fonctions anonymes qui peuvent **capturer** leur environnement ; elles s'utilisent avec les algorithmes de la bibliothèque standard.
- Les **espaces de noms** évitent les conflits ; l'**espace anonyme** rend des fonctions internes à un fichier.

4.25 Glossaire bilingue du chapitre

Français	Anglais	Définition courte
Argument	Argument	Valeur transmise lors d'un appel
Argument par défaut	Default argument	Valeur utilisée quand l'argument est omis

Français	Anglais	Définition courte
Cadre d'appel	Stack frame	Zone de la pile réservée à un appel de fonction
Capture	Capture	Accès d'une lambda à une variable extérieure
Cas de base	Base case	Condition d'arrêt d'une récursion
Déclaration	Declaration	Annonce d'une fonction sans son corps
Définition	Definition	Fonction complète, avec son corps
Élision de copie	Copy elision / RVO	Suppression d'une copie par le compilateur
Espace de noms	Namespace	Regroupement de noms
Espace de noms anonyme	Unnamed namespace	Espace dont les éléments sont internes au fichier
Fermeture	Closure	Objet créé par une lambda, contenant ses captures
Fonction pure	Pure function	Fonction sans effet de bord, dépendant de ses seuls paramètres
Lambda	Lambda expression	Fonction anonyme définie dans une expression
Liaison structurée	Structured binding	Décomposition d'un objet en variables nommées
Paramètre	Parameter	Variable qui reçoit un argument
Pile d'appels	Call stack	Empilement des cadres des fonctions en cours
Postcondition	Postcondition	Garantie offerte par la fonction après l'appel
Précondition	Precondition	Condition exigée de l'appelant avant l'appel
Récursivité	Recursion	Fonction qui s'appelle elle-même

Français	Anglais	Définition courte
Référence	Reference	Alias d'une variable existante
Référence pendante	Dangling reference	Référence vers un objet détruit
Résolution de surcharge	Overload resolution	Choix de la surcharge appelée
Signature	Signature	Nom et types des paramètres d'une fonction
Surcharge	Overloading	Plusieurs fonctions de même nom

4.26 Exercices corrigés

Exercice 1 — Choisir le mode de passage

Proposez la déclaration la plus adaptée pour chaque fonction.

1. `estPair` : teste si un entier est pair.
2. `afficherReleve` : affiche le nom d'un étudiant et son vecteur de notes.
3. `plafonner` : ramène à 20 toutes les notes d'un vecteur qui dépassent 20.
4. `normaliserNom` : produit une version en majuscules d'un nom, sans modifier l'original.
5. `echanger` : échange le contenu de deux `double`.
6. `lireNotes` : lit `n` notes au clavier.

Corrigé

```cpp
[[nodiscard]] bool estPair(int n);                                          // 1. petit type, entrée
void afficherReleve(const std::string& nom, const std::vector<double>& notes);  // 2. lecture de gros types
void plafonner(std::vector<double>& notes);                                 // 3. entrée-sortie
[[nodiscard]] std::string normaliserNom(std::string_view nom);              // 4. entrée en vue, résultat renvoyé
void echanger(double& a, double& b);                                        // 5. deux entrées-sorties
[[nodiscard]] std::vector<double> lireNotes(int n);                         // 6. résultat renvoyé
```

Pour 4, `std::string_view` accepte aussi bien un `std::string` qu'un littéral sans copie ; le résultat est une nouvelle chaîne, renvoyée par valeur.

Exercice 2 — Résolution de surcharge

Avec les déclarations suivantes, indiquez quelle fonction est appelée, ou si l'appel est invalide.

```cpp
void g(int);
void g(double);
void g(const std::string&);
void h(int, double = 1.0);
void h(double);
```

	Appel
a	`g(42)`
b	`g(4.2f)`
c	`g('x')`
d	`g("texte")`

	Appel
e	`g(42u)`
f	`h(3)`
g	`h(3.0)`

Corrigé

	Résultat	Raison
a	`g(int)`	Correspondance exacte
b	`g(double)`	Promotion `float` → `double`
c	`g(int)`	Promotion `char` → `int`
d	`g(const std::string&)`	Seule candidate viable : conversion définie par l'utilisateur `const char*` → `std::string`
e	**Ambigu**	`unsigned` → `int` et `unsigned` → `double` sont deux conversions standard
f	`h(int, double)`	Correspondance exacte pour `3` (le second paramètre prend sa valeur par défaut), contre une conversion pour `h(double)`
g	`h(double)`	Correspondance exacte, contre une conversion `double` → `int`

Exercice 3 — Trouver les bugs

```cpp
// a)
void echanger(double a, double b)
{
    double tmp = a;
    a = b;
    b = tmp;
}

// b)
const std::vector<double>& notesParDefaut()
{
    std::vector<double> v{10.0, 10.0};
    return v;
}

// c)  dans outils.hpp puis outils.cpp
double arrondir(double valeur, int decimales = 2);
double arrondir(double valeur, int decimales = 2) { /* ... */ }

// d)
int somme(int n)
{
    return n + somme(n - 1);
}

// e)
auto multiplicateur(double facteur)
{
    return [&facteur](double x) { return x * facteur; };
}
```

Corrigé

- **a)** Passage par valeur : la fonction échange des **copies**, les variables de l'appelant ne changent pas. Correction : `void echanger(double& a, double& b)`. En pratique, on utilise `std::swap(a, b)` de `<utility>`.
- **b)** Référence vers une variable locale détruite. Correction : renvoyer par valeur, `std::vector<double> notesParDefaut()`.
- **c)** L'argument par défaut est répété dans la définition : erreur de compilation. Correction : l'écrire uniquement dans la déclaration.
- **d)** Pas de cas de base : récursion infinie, débordement de pile. Correction : `if (n <= 0) { return 0; }` au début.
- **e)** `facteur` est un paramètre, donc une variable locale, détruite au retour de `multiplicateur` ; la lambda renvoyée garde une référence pendante. Correction : `[facteur]`.

Exercice 4 — Fonctions `constexpr` et tests à la compilation

Écrivez deux fonctions `constexpr` :

1. `puissance(double base, int exposant)` par l'algorithme d'exponentiation rapide récursif : $x^n = (x^{n/2})^2$ si n est pair, $x \cdot x^{n-1}$ sinon ;
2. `surVingt(double note, double bareme)` qui convertit une note sur un barème quelconque en note sur 20.

Vérifiez-les avec `static_assert`.

Corrigé

```cpp
constexpr double puissance(double base, int exposant)
{
    if (exposant == 0) {
        return 1.0;
    }
    if (exposant < 0) {
        return 1.0 / puissance(base, -exposant);
    }
    if (exposant % 2 == 0) {
        const double moitie = puissance(base, exposant / 2);
        return moitie * moitie;
    }
    return base * puissance(base, exposant - 1);
}

constexpr double surVingt(double note, double bareme)
{
    return note * 20.0 / bareme;
}

static_assert(puissance(2.0, 10) == 1024.0);
static_assert(puissance(3.0, 0) == 1.0);
static_assert(puissance(2.0, -2) == 0.25);
static_assert(surVingt(45.0, 60.0) == 15.0);
static_assert(surVingt(8.0, 10.0) == 16.0);
```

L'exponentiation rapide effectue environ $\log_2(n)$ multiplications au lieu de n. Les comparaisons exactes de réels sont acceptables ici, car ces valeurs particulières sont représentées exactement en binaire.

Exercice 5 — Lambdas

Soit `std::vector<double> notes{12.0, 8.5, 17.0, 14.25, 6.0, 10.0, 19.5};`. À l'aide de lambdas et des algorithmes `std::ranges`, écrivez le code qui :

1. compte les notes strictement inférieures à un seuil `seuilRattrapage` lu au clavier ;
2. teste s'il existe au moins une note supérieure ou égale à 19 ;
3. ajoute un bonus de 0,5 point à chaque note, sans dépasser 20 ;
4. trie les notes par ordre décroissant ;
5. calcule, avec une seule lambda et `std::ranges::for_each`, la somme et le nombre de notes validées (≥ 10).

Corrigé

```cpp
#include <algorithm>
#include <iostream>
#include <print>
#include <vector>

int main()
{
    std::vector<double> notes{12.0, 8.5, 17.0, 14.25, 6.0, 10.0, 19.5};

    double seuilRattrapage{};
    std::cin >> seuilRattrapage;

    // 1.
    const auto nbRattrapage = std::ranges::count_if(
        notes, [seuilRattrapage](double n) { return n < seuilRattrapage; });

    // 2.
    const bool excellence = std::ranges::any_of(notes, [](double n) { return n >= 19.0; });

    // 3.
    constexpr double bonus = 0.5;
    std::ranges::for_each(notes, [](double& n) { n = std::min(n + bonus, 20.0); });

    // 4.
    std::ranges::sort(notes, [](double a, double b) { return a > b; });

    // 5.
    double sommeValidees{0.0};
    int nbValidees{0};
    std::ranges::for_each(notes, [&sommeValidees, &nbValidees](double n) {
```

```
        if (n >= 10.0) {
            sommeValidees += n;
            ++nbValidees;
        }
    });

    std::println("À rattraper : {}, excellence : {}", nbRattrapage, excellence);
    std::println("Validées : {} (somme {})", nbValidees, sommeValidees);
}
```

Remarques :

- en 3, la lambda reçoit `double&` pour **modifier** les éléments ; `bonus` n'a pas besoin d'être capturé car c'est une constante `constexpr` : une lambda peut utiliser une variable `constexpr` de son environnement sans la capturer ;
- en 4, on aurait aussi pu écrire `std::ranges::sort(notes, std::greater{});` ;
- en 5, les deux accumulateurs sont capturés **par référence** pour être modifiés.

Exercice 6 — Structure de retour et liaisons structurées

Un matricule de l'UIN a la forme `"UIN-2026-001"`. Écrivez une fonction `analyserMatricule` qui renvoie une structure contenant le préfixe (`"UIN"`), l'année (`2026`, entier) et le numéro (`1`, entier). On suppose le format toujours correct. Affichez le résultat à l'aide d'une liaison structurée.

Corrigé

```cpp
#include <print>
#include <string>
#include <string_view>

struct InfosMatricule {
    std::string prefixe;
    int annee{};
    int numero{};
};

[[nodiscard]] InfosMatricule analyserMatricule(std::string_view matricule)
{
    const auto tiret1 = matricule.find('-');
    const auto tiret2 = matricule.find('-', tiret1 + 1);

    const std::string prefixe{matricule.substr(0, tiret1)};
    const std::string annee{matricule.substr(tiret1 + 1, tiret2 - tiret1 - 1)};
    const std::string numero{matricule.substr(tiret2 + 1)};

    return InfosMatricule{prefixe, std::stoi(annee), std::stoi(numero)};
}

int main()
{
    const auto [prefixe, annee, numero] = analyserMatricule("UIN-2026-001");
    std::println("Établissement : {}, promotion : {}, numéro : {}", prefixe, annee, numero);
}
```

`std::string_view::substr` renvoie une `string_view` (sans copie) ; on la convertit en `std::string` pour la stocker dans la structure et pour

`std::stoi`. Le cas d'un matricule mal formé sera traité au chapitre 15.

4.27 Travaux pratiques — TP 4 : carnet de notes en mode menu

Objectif : structurer un programme entièrement en fonctions, avec les bons modes de passage, des lambdas et un espace de noms.

Énoncé

Écrire un programme qui gère le carnet de notes d'une UE, avec le menu suivant :

```
=== Carnet de notes ===
1. Ajouter une note
2. Afficher les notes
3. Modifier une note
4. Supprimer une note
5. Trier les notes (croissant / décroissant)
6. Statistiques
7. Appliquer un bonus
0. Quitter
```

Contraintes :

1. Tout le code, sauf `main`, est dans l'espace de noms `carnet`.
2. Chaque option du menu correspond à **une** fonction.
3. Les fonctions de **calcul** (statistiques, bonus) n'effectuent **aucune** entrée-sortie.
4. Les modes de passage respectent la section 4.7.4.

5. Les statistiques (minimum, maximum, moyenne, nombre de validées) sont renvoyées dans une structure et affichées grâce à une liaison structurée.
6. Le tri et le bonus utilisent des lambdas.
7. Les fonctions dont le résultat compte sont `[[nodiscard]]`.
8. Réutiliser `lireEntier` et `lireNote` du chapitre 3.

Éléments de correction

```cpp
#include <algorithm>
#include <format>
#include <print>
#include <string_view>
#include <vector>

// lireEntier et lireNote du chapitre 3, placées ici ou incluses

namespace carnet {

enum class Ordre { Croissant, Decroissant };

struct Statistiques {
    double minimum{};
    double maximum{};
    double moyenne{};
    int nbValidees{};
};

// ---------- Calculs : aucune entrée-sortie ----------

/// @pre notes n'est pas vide
[[nodiscard]] Statistiques calculerStatistiques(const std::vector<double>& notes)
{
    const auto [mini, maxi] = std::ranges::minmax(notes);
    double somme{0.0};
    int validees{0};
    for (double n : notes) {
        somme += n;
        if (n >= 10.0) {
```

```
            ++validees;
        }
    }
    return {mini, maxi, somme / static_cast<double>(notes.size()), validees};
}

void appliquerBonus(std::vector<double>& notes, double bonus)
{
    std::ranges::for_each(notes, [bonus](double& n) { n = std::clamp(n + bonus, 0.0, 20.0); });
}

void trier(std::vector<double>& notes, Ordre ordre)
{
    if (ordre == Ordre::Croissant) {
        std::ranges::sort(notes);
    } else {
        std::ranges::sort(notes, [](double a, double b) { return a > b; });
    }
}

// ---------- Interface utilisateur ----------

void afficherNotes(const std::vector<double>& notes)
{
    if (notes.empty()) {
        std::println("Aucune note.");
        return;
    }
    for (std::size_t i{0}; i < notes.size(); ++i) {
        std::println("{:>3}. {:6.2f}", i + 1, notes[i]);
    }
```

```cpp
}

void ajouter(std::vector<double>& notes)
{
    notes.push_back(lireNote("Note à ajouter : "));
}

void modifier(std::vector<double>& notes)
{
    if (notes.empty()) {
        std::println("Aucune note à modifier.");
        return;
    }
    afficherNotes(notes);
    const int rang = lireEntier("Numéro de la note : ", 1, static_cast<int>(notes.size()));
    notes[static_cast<std::size_t>(rang - 1)] = lireNote("Nouvelle valeur : ");
}

void supprimer(std::vector<double>& notes)
{
    if (notes.empty()) {
        std::println("Aucune note à supprimer.");
        return;
    }
    afficherNotes(notes);
    const int rang = lireEntier("Numéro de la note : ", 1, static_cast<int>(notes.size()));
    notes.erase(notes.begin() + (rang - 1));
}

void afficherStatistiques(const std::vector<double>& notes)
{
```

```
    if (notes.empty()) {
        std::println("Aucune note.");
        return;
    }
    const auto [mini, maxi, moyenne, validees] = calculerStatistiques(notes);
    std::println("Min {:.2f} | Max {:.2f} | Moyenne {:.2f} | Validées {}/{}",
                 mini, maxi, moyenne, validees, notes.size());
}

[[nodiscard]] int afficherMenu()
{
    std::println("\n=== Carnet de notes ===");
    std::println("1. Ajouter une note\n2. Afficher les notes\n3. Modifier une note");
    std::println("4. Supprimer une note\n5. Trier les notes\n6. Statistiques");
    std::println("7. Appliquer un bonus\n0. Quitter");
    return lireEntier("Votre choix : ", 0, 7);
}

} // namespace carnet

int main()
{
    std::vector<double> notes;
    int choix{};
    do {
        choix = carnet::afficherMenu();
        switch (choix) {
        case 1: carnet::ajouter(notes); break;
        case 2: carnet::afficherNotes(notes); break;
        case 3: carnet::modifier(notes); break;
        case 4: carnet::supprimer(notes); break;
```

```
        case 5: {
            const int sens = lireEntier("1. Croissant  2. Décroissant : ", 1, 2);
            carnet::trier(notes, sens == 1 ? carnet::Ordre::Croissant : carnet::Ordre::Decroissant);
            break;
        }
        case 6: carnet::afficherStatistiques(notes); break;
        case 7: carnet::appliquerBonus(notes, lireNote("Bonus : ")); break;
        default: break;
        }
    } while (choix != 0);
}
```

Remarques :

- `notes.erase(notes.begin() + (rang - 1))` utilise un **itérateur** (chapitre 18) ; retenez pour l'instant que `notes.begin() + k` désigne l'élément d'indice `k` ;
- `std::clamp(x, bas, haut)` borne une valeur dans un intervalle ;
- dans le `case 5`, les accolades créent une portée pour la variable `sens` : sans elles, déclarer une variable initialisée dans un `case` est une erreur de compilation ;
- les fonctions de calcul (`calculerStatistiques`, `appliquerBonus`, `trier`) peuvent être testées sans clavier ni écran : c'est l'intérêt de la séparation calcul / interface.

4.28 Fil rouge — Module de calculs de UIN-Campus

Nous ajoutons un module de **calculs académiques**, nous améliorons la **saisie** avec des structures de retour, et `main` apprend à lire un argument de la ligne de commande.

4.28.1 Nouvelle arborescence

```
uin-campus/
├── include/uincampus/
│   ├── version.hpp
│   ├── regles.hpp
│   ├── saisie.hpp
│   └── calculs.hpp       ← nouveau
├── src/
│   ├── version.cpp
│   ├── regles.cpp
│   ├── saisie.cpp        ← modifié
│   └── calculs.cpp       ← nouveau
└── app/
    └── main.cpp          ← modifié
```

4.28.2 `include/uincampus/calculs.hpp`

```cpp
#pragma once

#include "uincampus/regles.hpp"

#include <vector>

namespace uin {

/// Indicateurs statistiques d'une série de notes.
struct Statistiques {
    double minimum{};
    double maximum{};
    double moyenne{};
    double ecartType{};
};

/// Somme des valeurs.
[[nodiscard]] double somme(const std::vector<double>& valeurs);

/// Moyenne arithmétique.
/// @pre notes n'est pas vide
[[nodiscard]] double moyenne(const std::vector<double>& notes);

/// Moyenne pondérée par les coefficients.
/// @pre notes n'est pas vide, notes et coefficients ont la même taille,
///      la somme des coefficients est strictement positive
[[nodiscard]] double moyenne(const std::vector<double>& notes,
                             const std::vector<double>& coefficients);

/// Minimum, maximum, moyenne et écart-type.
```

```
/// @pre notes n'est pas vide
[[nodiscard]] Statistiques calculerStatistiques(const std::vector<double>& notes);

/// Nombre de notes supérieures ou égales au seuil.
[[nodiscard]] int compterValidees(const std::vector<double>& notes,
                                  double seuil = seuilValidation);

/// Arrondi au nombre de décimales demandé.
[[nodiscard]] double arrondir(double valeur, int decimales = 2);

} // namespace uin
```

On y retrouve les notions du chapitre : **surcharge** (deux `moyenne`), **arguments par défaut** (`seuil`, `decimales`), **structure de retour**, `[[nodiscard]]`, **préconditions** documentées.

4.28.3 `src/calculs.cpp`

```cpp
#include "uincampus/calculs.hpp"

#include <algorithm>
#include <cassert>
#include <cmath>
#include <numeric>

namespace uin {

double somme(const std::vector<double>& valeurs)
{
    return std::accumulate(valeurs.begin(), valeurs.end(), 0.0);
}

double moyenne(const std::vector<double>& notes)
{
    assert(!notes.empty() && "moyenne() : aucune note");
    return somme(notes) / static_cast<double>(notes.size());
}

double moyenne(const std::vector<double>& notes, const std::vector<double>& coefficients)
{
    assert(!notes.empty() && "moyenne() : aucune note");
    assert(notes.size() == coefficients.size() && "moyenne() : tailles différentes");

    const double sommeCoefficients = somme(coefficients);
    assert(sommeCoefficients > 0.0 && "moyenne() : somme des coefficients nulle");

    const double sommePonderee =
        std::inner_product(notes.begin(), notes.end(), coefficients.begin(), 0.0);
```

```cpp
    return sommePonderee / sommeCoefficients;
}

Statistiques calculerStatistiques(const std::vector<double>& notes)
{
    assert(!notes.empty() && "calculerStatistiques() : aucune note");

    const auto [mini, maxi] = std::ranges::minmax(notes);
    const double moy = moyenne(notes);
    const double sommeCarres = std::accumulate(
        notes.begin(), notes.end(), 0.0,
        [moy](double total, double n) { return total + (n - moy) * (n - moy); });

    return Statistiques{
        .minimum = mini,
        .maximum = maxi,
        .moyenne = moy,
        .ecartType = std::sqrt(sommeCarres / static_cast<double>(notes.size())),
    };
}

int compterValidees(const std::vector<double>& notes, double seuil)
{
    return static_cast<int>(
        std::ranges::count_if(notes, [seuil](double n) { return n >= seuil; }));
}

double arrondir(double valeur, int decimales)
{
    const double facteur = std::pow(10.0, decimales);
    return std::round(valeur * facteur) / facteur;
```

```
}

} // namespace uin
```

Points à remarquer :

- `std::inner_product` (de `<numeric>`) calcule directement Σ note$_i$ × coef$_i$;
- `std::accumulate` reçoit une **lambda** qui capture `moy` par valeur, pour accumuler les carrés des écarts ;
- la structure est construite avec des **initialiseurs désignés** (`.minimum = mini`, C++20), qui rendent chaque champ explicite (chapitre 8) ;
- les préconditions sont vérifiées par `assert` : une erreur de programmation (appel avec un vecteur vide) est détectée immédiatement en Debug.

4.28.4 `src/saisie.cpp` : des structures plutôt que des paramètres de sortie

Au chapitre 3, les fonctions internes renvoyaient un `bool` et remplissaient un paramètre de sortie (`int& valeur`). Nous appliquons la règle « résultat par `return` » :

```cpp
#include "uincampus/saisie.hpp"
#include "uincampus/regles.hpp"

#include <iostream>
#include <sstream>

namespace uin {

namespace {

struct EntierLu {
    bool valide{false};
    int valeur{0};
};

struct ReelLu {
    bool valide{false};
    double valeur{0.0};
};

std::string lireLigne(std::string_view invite)
{
    std::cout << invite;
    std::string ligne;
    std::getline(std::cin, ligne);
    return ligne;
}

[[nodiscard]] EntierLu versEntier(const std::string& texte)
{
```

```
    std::istringstream flux{texte};
    int valeur{};
    char reste{};
    const bool valide = (flux >> valeur) && !(flux >> reste);
    return {valide, valeur};
}

[[nodiscard]] ReelLu versReel(std::string texte)
{
    for (char& c : texte) {
        if (c == ',') {
            c = '.';
        }
    }
    std::istringstream flux{texte};
    double valeur{};
    char reste{};
    const bool valide = (flux >> valeur) && !(flux >> reste);
    return {valide, valeur};
}

} // namespace

std::string lireTexte(std::string_view invite)
{
    return lireLigne(invite);
}

int lireEntier(std::string_view invite, int min, int max)
{
    while (std::cin) {
```

```
        const auto [valide, valeur] = versEntier(lireLigne(invite));
        if (!valide) {
            std::cout << "  Saisie invalide : entrez un nombre entier.\n";
        } else if (valeur < min || valeur > max) {
            std::cout << "  La valeur doit être comprise entre " << min << " et " << max << ".\n";
        } else {
            return valeur;
        }
    }
    return min;
}

double lireNote(std::string_view invite)
{
    while (std::cin) {
        const auto [valide, note] = versReel(lireLigne(invite));
        if (!valide) {
            std::cout << "  Saisie invalide : entrez un nombre (ex. 12.5 ou 12,5).\n";
        } else if (!estNoteValide(note)) {
            std::cout << "  La note doit être comprise entre " << noteMin << " et " << noteMax << ".\n";
        } else {
            return note;
        }
    }
    return noteMin;
}

} // namespace uin
```

La lecture devient plus claire : `const auto [valide, valeur] = versEntier(...)`. Au chapitre 15, ces petites structures seront remplacées par

`std::optional`, conçu exactement pour ce besoin.

4.28.5 `app/main.cpp`

```cpp
#include "uincampus/calculs.hpp"
#include "uincampus/regles.hpp"
#include "uincampus/saisie.hpp"
#include "uincampus/version.hpp"

#include <cstdlib>
#include <format>
#include <print>
#include <string>
#include <string_view>
#include <vector>

namespace {

void afficherLigne(std::string_view libelle, std::string_view valeur)
{
    std::println("{:<24}{:>20}", libelle, valeur);
}

} // namespace

int main(int argc, char* argv[])
{
    if (argc > 1 && std::string_view{argv[1]} == "--version") {
        std::println("uin-campus {}", uin::version());
        return EXIT_SUCCESS;
    }

    std::println("UIN-Campus {} — Relevé d'un étudiant\n", uin::version());
```

```
    const std::string nom = uin::lireTexte("Nom de l'étudiant : ");
    const int nbUE = uin::lireEntier("Nombre d'UE (1 à 30) : ", 1, 30);

    std::vector<double> notes;
    std::vector<double> coefficients;
    for (int i{1}; i <= nbUE; ++i) {
        notes.push_back(uin::lireNote(std::format("  Note de l'UE {} : ", i)));
        coefficients.push_back(
            static_cast<double>(uin::lireEntier(std::format("  Coefficient de l'UE {} : ", i), 1, 10)));
    }

    const double moyenneGenerale = uin::arrondir(uin::moyenne(notes, coefficients));
    const auto [mini, maxi, moyenneSimple, ecartType] = uin::calculerStatistiques(notes);
    const int validees = uin::compterValidees(notes);

    std::println("\n{:-<44}", "");
    afficherLigne("Étudiant", nom);
    afficherLigne("Moyenne pondérée", std::format("{:.2f}", moyenneGenerale));
    afficherLigne("Moyenne simple", std::format("{:.2f}", moyenneSimple));
    afficherLigne("Note min / max", std::format("{:.2f} / {:.2f}", mini, maxi));
    afficherLigne("Écart-type", std::format("{:.2f}", ecartType));
    afficherLigne("UE validées", std::format("{} / {}", validees, nbUE));
    afficherLigne("Mention", uin::libelle(uin::mentionPour(moyenneGenerale)));
    std::println("{:-<44}", "");

    return EXIT_SUCCESS;
}
```

Nouveautés dans `main` :

- lecture de l'argument `--version` ;
- moyenne **pondérée** grâce à la surcharge de `moyenne` ;
- statistiques récupérées par **liaison structurée** ;
- une fonction utilitaire `afficherLigne` dans un **espace de noms anonyme**, qui supprime la répétition du format d'affichage ;
- la mention est désormais attribuée sur la moyenne **pondérée**, conformément aux règles académiques.

4.28.6 Mise à jour du `CMakeLists.txt`

```
add_library(uincampus_core
    src/version.cpp
    src/regles.cpp
    src/saisie.cpp
    src/calculs.cpp)
```

4.28.7 Construire, tester, versionner

```
cmake --build --preset debug
./build/debug/uin-campus --version
./build/debug/uin-campus
```

Vérifiez à la main un cas simple : notes 12 (coef 2) et 15 (coef 1) doivent donner une moyenne pondérée de 13,00 et une moyenne simple de 13,50. Puis :

```
clang-format -i include/uincampus/*.hpp src/*.cpp app/*.cpp
git add .
git commit -m "Ajouter le module de calculs et la moyenne pondérée"
```

Au chapitre 5, nous étudierons la mémoire, les pointeurs et les tableaux, ce qui permettra notamment de comprendre `argv` et d'introduire `std::span`.

4.29 Pour aller plus loin

- C++ Core Guidelines, section **F** (*Functions*), en particulier F.15 à F.21 sur le passage des paramètres et les valeurs de retour.
- cppreference.com : pages *Overload resolution*, *Lambda expressions*, *Namespaces*, *Structured binding declaration*.
- Scott Meyers, *Effective Modern C++* : items 31 à 34 sur les lambdas.
- learncpp.com : chapitres sur les fonctions, les références et les lambdas.

Chapitre suivant : ***Chapitre 5 — Mémoire, pointeurs et références***